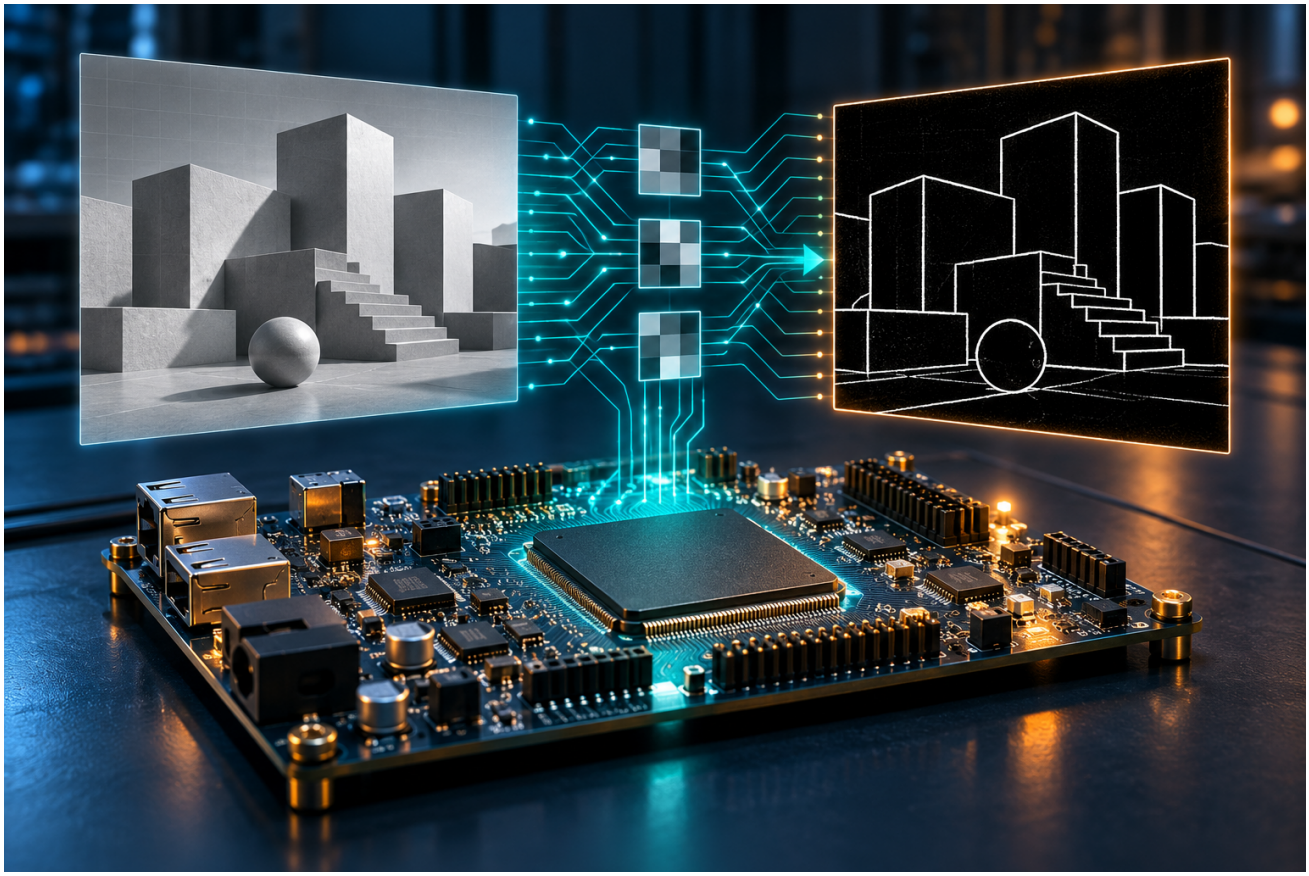
From pixels to edges

Understanding the Sobel project, the hardware pipeline, and your role in system integration.



THE PROJECT IN ONE SENTENCE

Send a grayscale image from a computer to an FPGA, process it with Sobel edge detection, store the result, display it, and send it back to the computer for a pixel-by-pixel correctness check.

Development order: correct Sobel first, a correct fixed noise filter next, and offline parameter optimization after that.

A complete English version of the project explanation, with generated conceptual illustrations.

What are we processing?

A grayscale image is a table of numbers. Each number represents the brightness of one pixel: 0 is black, 255 is white, and the values between them are shades of gray.

A 256 x 256 image contains 65,536 pixels. At 8 bits per pixel, the raw image requires 64 KiB. During transfer, the pixels are sent in **row-major order**: the first row from left to right, then the second row, and so on.

How Sobel finds an edge

Sobel looks for changes in brightness between neighboring pixels. An area with the same gray level usually contains no edge; a transition from a dark area to a bright one can indicate an edge.

For each calculation, it needs a 3 x 3 neighborhood:

```
p11 p12 p13
p21 p22 p23
p31 p32 p33
```

The calculation produces a weighted left-to-right difference, **Gx**, and a weighted top-to-bottom difference, **Gy**. The project reference uses:

```
magnitude = |Gx| + |Gy|
output = 255 if magnitude > threshold
output = 0 otherwise
```

The output is therefore a black-and-white edge image. This is the Sobel calculation implemented in `sobel_math.v`.

> **The correctness contract:** Python and RTL must use the same arithmetic, threshold, and image-border policy. Otherwise, their results cannot be compared fairly.

What exists, and what will you build?

The existing Sobel reference provides a processing core and a simulation workflow:

Component	Purpose
line_buffer	Collects the pixel stream into a 3 x 3 window.
sobel_math	Calculates edges from the window.
sobel_top	Connects the window generator and Sobel calculation.
tb_sobel	Feeds an image into the simulation and saves the output.
Python scripts	Convert an image to HEX and reconstruct an image from the output.

Adding UART, output-image memory, VGA, and the connections to the FPGA board is part of your team's work.

The system reference has two different paths

The accompanying system example can help you build the surrounding hardware. Its `src2` path receives a still image through UART; its `src1` path is for a camera and live video. These are different use cases.

The clock generators and constraints in that example are tied to the hardware it was built for. They must be adapted to your board.

What the existing presentation contributes

The existing project presentation explains window construction, the Sobel calculation, and the test flow. Treat its resource tables as reference figures. Your project's resource usage must come from implementing your own design on your own FPGA board.

> **Practical boundary:** the available processing core is the starting point. A complete board-level image-processing system still needs communication, storage, display, control, and verification.

How does an image move through the system?

A sensible initial architecture is the following. Final memory and clock choices should be fixed once the FPGA board is known.

PC image preparation → UART RX → Input FIFO
→ Line buffers + 3 x 3 window → Sobel + threshold
→ Output frame buffer

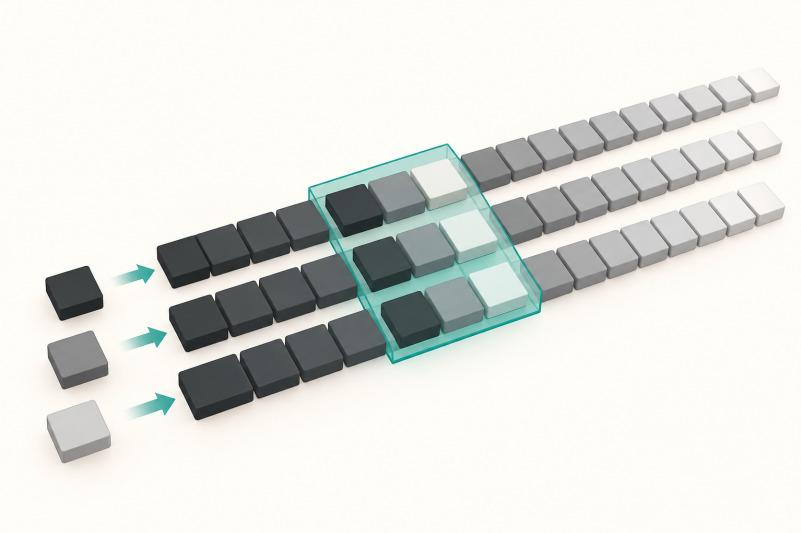
From the frame buffer: **VGA display** and **UART TX** → **PC comparison**

Block	Why it is needed
UART RX	Receives bits from the computer and assembles bytes. Initially, use one byte per grayscale pixel.
Input FIFO	Provides temporary storage and preserves data order when reception and processing do not advance together.
Line buffers	Keep the previous two rows and combine them with the current row to provide nine neighboring pixels.
Sobel	Calculates the edge strength and applies the threshold.
Frame buffer	Stores the result so the display can read the image repeatedly.
VGA	Outputs the image using the display's required timing.
UART TX	Returns the result to the computer, which is especially useful for proving that the board works correctly.

FIFO, line buffer, and frame buffer have three different jobs, even though all three use storage.

If VGA and UART TX read the same memory, you must decide which block has a read port, or how a controller schedules their reads. This is one of your integration decisions.

Why do we need line buffers?



Pixels arrive one after another, but Sobel needs to see three rows at the same time. When a new pixel arrives, the line buffers supply the pixels above it from the previous two rows. Shift registers retain the last three columns in each row. Together, they form the 3 x 3 window.

This is the idea behind the FIFO and shift-register arrangement in the existing presentation.

A window must be correct in space

At the start of a new row, the last two columns of the old row must not become the left-hand neighbors of the first pixel in the new row. A continuous stream alone does not guarantee a valid image neighborhood.

You therefore need to track:

- Which row are we in?
- Which column are we in?
- Are all nine neighbors available?
- Which image coordinate does the output represent?

> **This is central to your work:** window validity must reflect image coordinates, not just the number of pixels that have arrived.

Illustration: a conceptual three-row pixel neighborhood. It is not a cycle-accurate circuit diagram.

Simulation and the actual FPGA

In simulation, the testbench plays the role of the outside world:

```
Image → HEX → Testbench → RTL
→ Output HEX → Python comparison
```

The testbench generates the clock and reset, supplies pixels, and collects the output. Operations such as `$readmemh` and `$fwrite` in this flow happen inside the simulation environment.

On the board, the design needs a physical input such as UART or a camera interface, and a physical output such as VGA or UART TX. The testbench itself is not loaded onto the FPGA.

The implementation flow

```
RTL → Synthesis → Place & Route
→ Timing checks → Bitstream → FPGA
```

Passing simulation demonstrates the design's behavior for the cases you tested. Passing implementation checks and running the design on the board demonstrate additional properties, including timing and actual hardware connections.

Latency is different from throughput

Term	Meaning
Latency	How long a pixel takes to produce its result.
Throughput	How quickly the system can accept or produce pixels.

The processing core may achieve one pixel per clock during continuous operation, while image transfer over UART is much slower.

Example: at 115,200 baud with 8N1 framing, each byte uses 10 serial bits. Transferring a raw 256 x 256 image takes approximately **5.7 seconds**, before protocol overhead.

Measure core processing speed and complete computer-to-result system speed separately.

Where do the noise filter and GA fit?



The proposed extension adds a smoothing filter before Sobel to reduce noise. A **Genetic Algorithm (GA)** in Python chooses the filter settings and threshold. Once selected, those settings are used by the RTL on the FPGA.

The idea in four steps

- 1. Try different filter settings and thresholds.
- 2. Measure the quality of the resulting edges.
- 3. Let the GA retain better settings and create new candidates.
- 4. Fix the selected settings and test their hardware implementation.

Under the current proposal, the FPGA performs image processing, while the parameter search happens offline.

Build in three stages: a correct Sobel core, then a correct fixed filter, then GA-based selection of settings.

The filter changes the architecture

A 3 x 3 filter followed by a 3 x 3 Sobel stage generally needs a second window built from the **filtered pixels**. The same nine raw pixels are not enough to execute these two consecutive stages directly.

Freely chosen filter weights can introduce multipliers and normalization arithmetic. Baraa and Ashraf need to agree with you on settings that can be implemented, including bit widths and rounding. The Python model must reproduce that arithmetic exactly.

Illustration: conceptual noisy input, smoothed input, and edge output. These panels are generated artwork, not measured results.

Your first responsibility: define interfaces early

Your earlier objection was correct: interfaces cannot wait until everyone has finished writing RTL. Define the specifications at the beginning, then check compliance when blocks are delivered.

The team needs a short agreement that covers the following decisions:

Decision	What must be specified
Image	Dimensions, grayscale format, and pixel order.
UART	Baud rate, message format, and how a frame begins and ends.
Clocks / reset	The clock used by each block, and what reset does.
Data interface	Data width and the meaning of valid and ready.
Memory	Read latency, addresses, and read/write signals.
Sobel	Threshold, arithmetic, and pipeline latency.
Borders	Ignore borders, use padding, or output zeros at borders?
Completion	Exactly when frame_done becomes true.

Example: a valid / ready interface

- `valid = 1`: the sender has a valid pixel.
- `ready = 1`: the receiver can accept it.
- A transfer occurs on a rising clock edge when both signals are 1.

> **Advance the pixel counter when a pixel is actually accepted.** Do not advance it on every clock: UART may take many clocks before producing another byte.

Not every block must use ready. A simpler interface is possible if the design guarantees that every pixel can be accepted. That guarantee must be explicit and tested.

Your second responsibility: integrate and verify

The top module instantiates blocks and connects signals. Your work is broader than wiring: the data, timing, and meaning of each signal must agree between blocks.

For example, if memory returns data one clock after a read request, `pixel_valid` must reach Sobel with the returned data, not with the request address. A one-clock mistake can shift the whole image.

Integrate in this order

Stage	What to prove
1. Testbench to Sobel core	Prove the arithmetic and window construction before introducing UART.
2. UART RX to FIFO	Prove byte order and that no bytes are lost.
3. FIFO to Sobel	Test gaps between incoming pixels.
4. Sobel to output memory	Prove that each result is written to the correct location.
5. Memory to TX / VGA	Prove readout and display behavior.
6. Complete system	Send an image in and receive a result that matches the golden model.

Each block owner should deliver the RTL and its own testbench. You own the system testbench and the compatibility review between blocks.

Make the system testbench self-checking

It must compare expected and actual results automatically, then report the error count and the locations of errors. An image that looks good, or a waveform alone, is not a sufficient correctness criterion.

> **The target:** the complete system accepts an image and produces the expected result, pixel by pixel, under the agreed interface and border rules.

Test cases and the border trap

Start with simple cases that isolate individual failure modes:

Test	What it reveals
Uniform image	The interior should contain no edges.
Vertical and horizontal lines	Correct Gx / Gy behavior and neighbor ordering.
Small numbered image	Correct construction of each 3 x 3 window.
Random gaps between pixels	Correct valid handling and counters.
Two consecutive images	Correct initialization at the start of a new frame.
Reset during operation	Return to a known state.
Real image	Pixel-by-pixel agreement with Python.

If the design supports backpressure, also pause the receiver temporarily and check that data is neither lost nor duplicated.

A concern in the existing implementation

The previous code review identified a point that needs checking: `window_valid` depends on passing a count of 515 pixels, without a condition that excludes the start of every row. The Python reconstruction also fills the end of the list with zeros. This does not automatically fix image borders or pixel coordinates.

These observations came from reading `line_buffer.v` and `hex_to_image.py`. No simulation was run during that review.

Count outputs from the chosen border policy

If a 256 x 256 image produces only interior window centers, the correct count is:

```
(256 - 2) x (256 - 2) = 64,516 output pixels
```

Producing an image with the original dimensions requires an explicit border policy. The figure of 65,021 outputs in the existing presentation does not, by itself, prove that the image windows are correct.